What Even *Is* a Program?

Class 1 — In-Class Work-Along Sheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Fill it in *as we go*. Attempt every question **before** we discuss it — your first guesses do not need to be right. Keep your **Terminal** open; some tasks ask you to run things using the files your instructor shares.

Part 1 — Languages, Files, and How Code Actually Runs

Q1. What programming languages have you *heard of*?

Q2. Which of these languages do you actually *know* (even a little)?

Q3. *(Answer this if you know more than one language.)* Do you find these languages **startlingly different**, or do they **share common things**? Elaborate.

ON THE BOARD — five "Hello World" programs

```
C            #include <stdio.h>
             int main() { printf("Hello, World!\n"); return 0; }

C++          #include <iostream>
             int main() { std::cout << "Hello, World!" << std::endl; return 0; }

Java         public class HelloWorld {
                 public static void main(String[] args) {
                     System.out.println("Hello, World!");
                 }
             }

Python       print("Hello, World!")

JavaScript   console.log("Hello, World!");
```

Q4. You just saw programs written in several languages. **What do you think they are doing?**

TRY THIS — run them yourself

Using the files + commands your instructor shares, run all five. Run commands look like:

```
C / C++ :  gcc hello.c -o hello   then   ./hello
Java    :  javac HelloWorld.java   then   java HelloWorld
Python  :  python3 hello.py
JavaScript: node hello.js
```

Now try running the **Python** program with the **Java** command.

Q5. Did you run all these programs? Is the output the **same**? What **varies** across them? And what happens when you try to run the **Python program with the Java command**?

Q6. A program is just text saved in a **file**, and a file name has **two parts** (e.g. `hello_world.py` → `hello_world` + `.py`). What are those two parts? Can the **extension** be anything random? **Try it:** change a Python file's extension to `.txt` and run it — write your observation.

Q7. *(Mini-research.)* You saw that Python is *flexible* about the file extension. Is **C++** the same? Research: **how does C++ respond when you run a file that does NOT have the `.cpp` extension?**

Q8. Summarize everything you have learned *so far* about what a "programming language" is.

Q9. What language does your **hardware** actually understand? And how does **Python code** (which is just text!) get converted into that language? *(Draw or describe.)*

```
+-------------------------------------------------------------------------+
|                                                                         |
|                                                                         |
|                                                                         |
+-------------------------------------------------------------------------+
```

Q10. In your own words, define each: - **bytecode:** _____ - **process:** _____ - **process ID (PID):** _____ - **who manages them / role of the Operating System in running your program:**

Q11. Look at the Python program your instructor ran (the one that imports other files). (a) How many **modules** does it import? _____ (b) Which files got their **bytecode** generated? _____

Q12. Was bytecode generated for the **top-level script you ran** itself? **Research why.**

Part 2 — Think Like a Programmer (meet Zog)



Zog is an alien who just landed from the planet **Zogland**. Zog wants to help — but Zog only understands its own language, **Zoglang**, and Zog is *relentlessly literal*: it does **exactly** what you write, top to bottom, and assumes nothing.

🪶 **The Golden Rule of Zog:** if the program does the wrong thing, Zog isn't dumb — *your instructions were.*

Zoglang — the whole language on one card

```
SPEAK & HEAR
  BLURT  <thing>           Zog says it out loud.                BLURT "Greetings!"   BLURT total
  LISTEN <name>            Zog listens to the human & stores it.  LISTEN name

VARIABLES  (a labelled box Zog keeps)
  GULP  <name>             Make a NEW empty box (no value yet).   GULP score
  GULP  <name> = <value>   Make a new box AND fill it.            GULP score = 0
  <name> = <expr>          Put the right-side result in the box.  score = score + 10

DECISIONS                     LOOP (for every item of a list)
  IF <condition>                FOR EACH <name> IN <list>
     ...                           ...
  OTHERWISE   (optional)        DONE
     ...
  DONE

LISTS    [4, 8, 15, 16]
MATHS    +  -  *  /  %        ( % = leftover of a division; 7 % 3 = 1 )
         + adds numbers, but ALSO joins text:  "Hi, " + name
COMPARE  ==  >  <  >=  <=     ( == asks "are these equal?" )
LOGIC    AND   OR   NOT
```

Write your Zoglang programs in the boxes. **Plan in plain English first, then translate.**

🟢 Warm-ups

S1 — "Zog Says Hi." Make Zog blurt a greeting to the class.

```
+------------------------------------------------------------------------+
|                                                                        |
|                                                                        |
+------------------------------------------------------------------------+
```

S2 — "What's Your Name?" Zog listens to a human's name, then greets them *by name*.

```
+------------------------------------------------------------------------+
|                                                                        |
|                                                                        |
+------------------------------------------------------------------------+
```

S3 — "Zog Does Sums." Zog listens to two numbers and blurts their total.

```
+------------------------------------------------------------------------+
|                                                                        |
|                                                                        |
+------------------------------------------------------------------------+
```

🟡 Now think before you write — the THINK → PLAN → TRANSLATE method

USE THIS FOR EVERY PROBLEM FROM HERE ON

① **THINK** — what's the *input*, what's the *output*? Restate the problem in your own words. ② **PLAN** — write the steps (the recipe / algorithm) in plain English. ③ **TRANSLATE** — turn each plan-step into Zoglang. ④ **CHECK** — dry-run it with a small example, the way Zog would.

M1 — "The Bigger Beast." Zog listens to two numbers and blurts the larger one.

PLAN (plain English): _____

_____

```
+------------------------------------------------------------------------+
|                                                                        |
|                                                                        |
|                                                                        |
+------------------------------------------------------------------------+
```

M2 — "Lucky or Spooky." Zog listens to a number; blurts `Lucky!` if it is **even**, `Spooky!` if **odd**.

```
+------------------------------------------------------------------------+
|                                                                        |
|                                                                        |
|                                                                        |
+------------------------------------------------------------------------+
```

M3 — "Roll Call." Zog has a crew list `["Zog", "Blip", "Nova"]`. For each name, blurt `Aboard: <name>`.

```
+------------------------------------------------------------------------+
|                                                                        |
|                                                                        |
|                                                                        |
+------------------------------------------------------------------------+
```

🔴 Challenges

H1 — "Treasure Tally." Zog's satchel holds gem values `[7, 2, 9, 4]`. Total them and blurt the sum.

PLAN:

_____

```
+------------------------------------------------------------------------+
|                                                                        |
|                                                                        |
|                                                                        |
|                                                                        |
+------------------------------------------------------------------------+
```

H2 — "The Biggest Gem." From the same satchel `[7, 2, 9, 4]`, find and blurt the **single largest** gem.

PLAN:

_____



Wrap-up (books closed!). In one or two sentences: what is the *difference* between the **source code** you write and the **process** that actually runs on the hardware?

_____

_____